МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Косарим П. Д.
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: Изучение основ 3D-графики: построение простых 3D-объектов, проекция на 2D-плоскость, работа с матрицами перспективы, ортографической проекции и аффинными преобразованиями с использованием API Vulkan, GLFW и ImGUI.
Вариант задания: вариант 1 — куб.

Метод решения

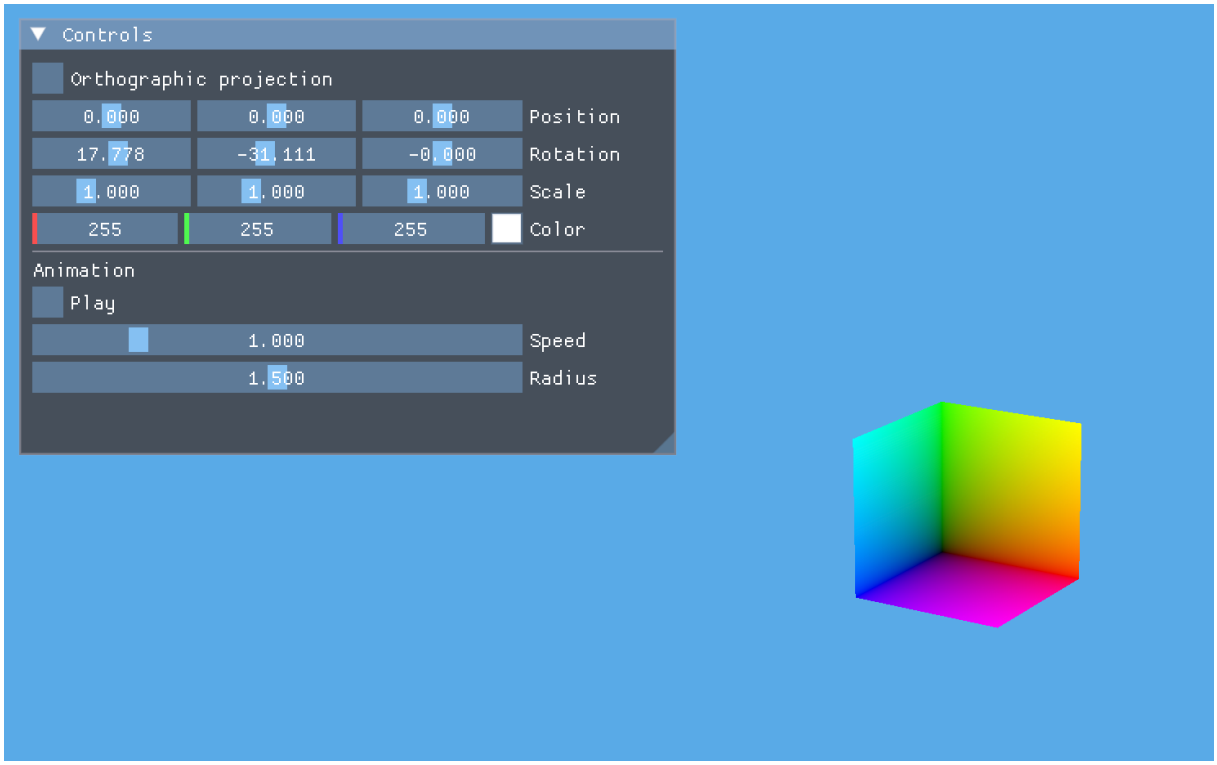
Для реализации использовался предоставленный стартовый код на C++ с использованием Vulkan. Были выполнены следующие шаги:

1. Построение геометрии: Куб задан с помощью 8 вершин (VkBuffer) и 36 индексов (VkBuffer), которые загружаются в память GPU с помощью библиотеки Vulkan Memory Allocator (VMA). Для отрисовки используется команда vkCmdDrawIndexed.
2. Матрицы трансформации: Для работы с 3D-графикой была подключена библиотека линейной алгебры glm. В вершинный шейдер передаются три матрицы
   (через UniformBufferObject и VkDescriptorSet):
   - model — отвечает за позицию, поворот и масштаб куба.
   - view — задает положение камеры (матрица вида).
   - proj — задает проекцию.
3. Дополнительные задания:
   - Задание 1 (Проекция): В ImGui добавлен чекбокс. В зависимости от его состояния вычисляется либо матрица перспективной проекции (glm::perspective), либо ортографической (glm::ortho).
   - Задание 2 (Трансформации): В ImGui добавлены слайдеры SliderFloat3, которые напрямую влияют на переменные позиции, поворота и масштаба. Эти переменные перемножаются в матрице model.
   - Задание 3 (Анимация): Реализован расчет времени между кадрами (delta time). Куб движется по круговой траектории (используя sin и cos от накопленного времени) и вращается вокруг своей оси. В ImGui добавлены чекбокс паузы и слайдеры для настройки скорости и радиуса траектории.

- Задание 4 (Изменение цвета): В ImGui добавлен ColorEdit3. Выбранный цвет передается в шейдер через vec4 в UniformBufferObject.
- Задание 5 (Процедурные цвета): Цвета вершин куба вычисляются динамически на основе их локальных координат. В C++ коде создана функция, которая нормализует координаты X, Y, Z в диапазон от 0 до 1 и назначает их как компоненты R, G, B соответственно. В шейдере выбранный пользователем цвет умножается на этот процедурный цвет вершины (fragColor = inColor * ubo.color.rgb).

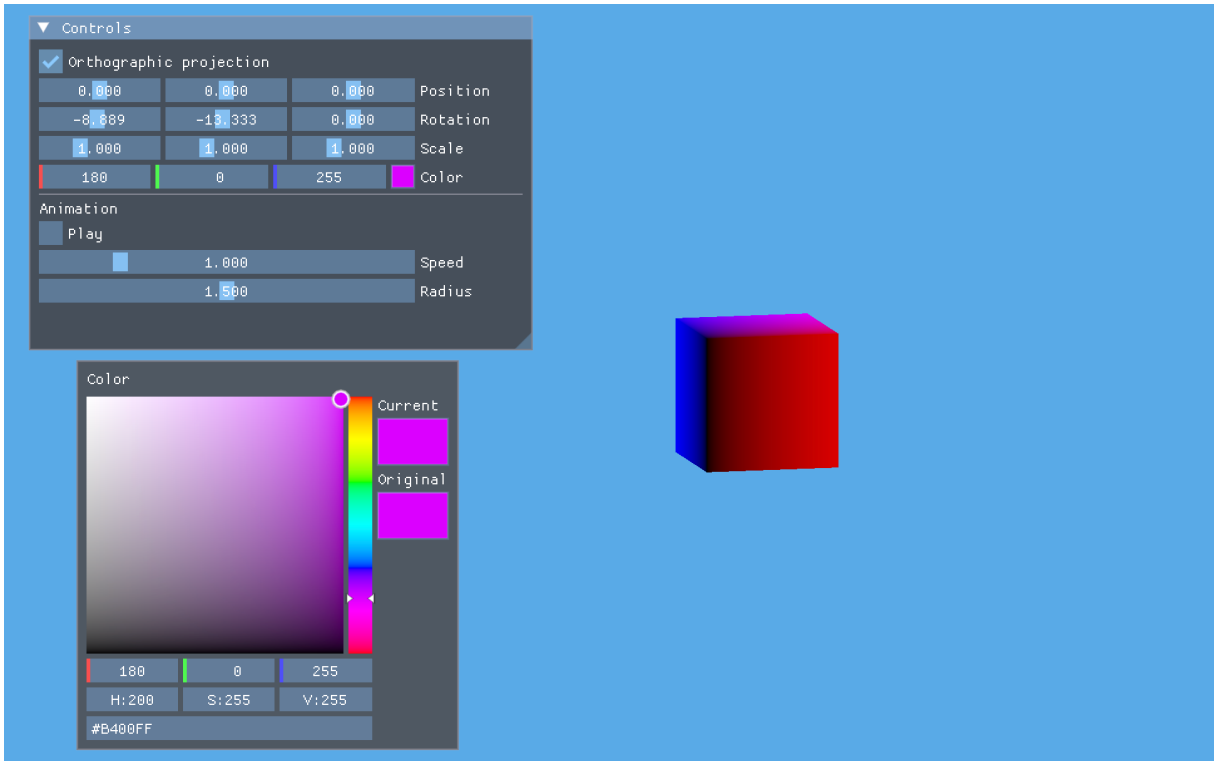
Результаты

Скриншот 1. Интерфейс управления и базовое отображение куба



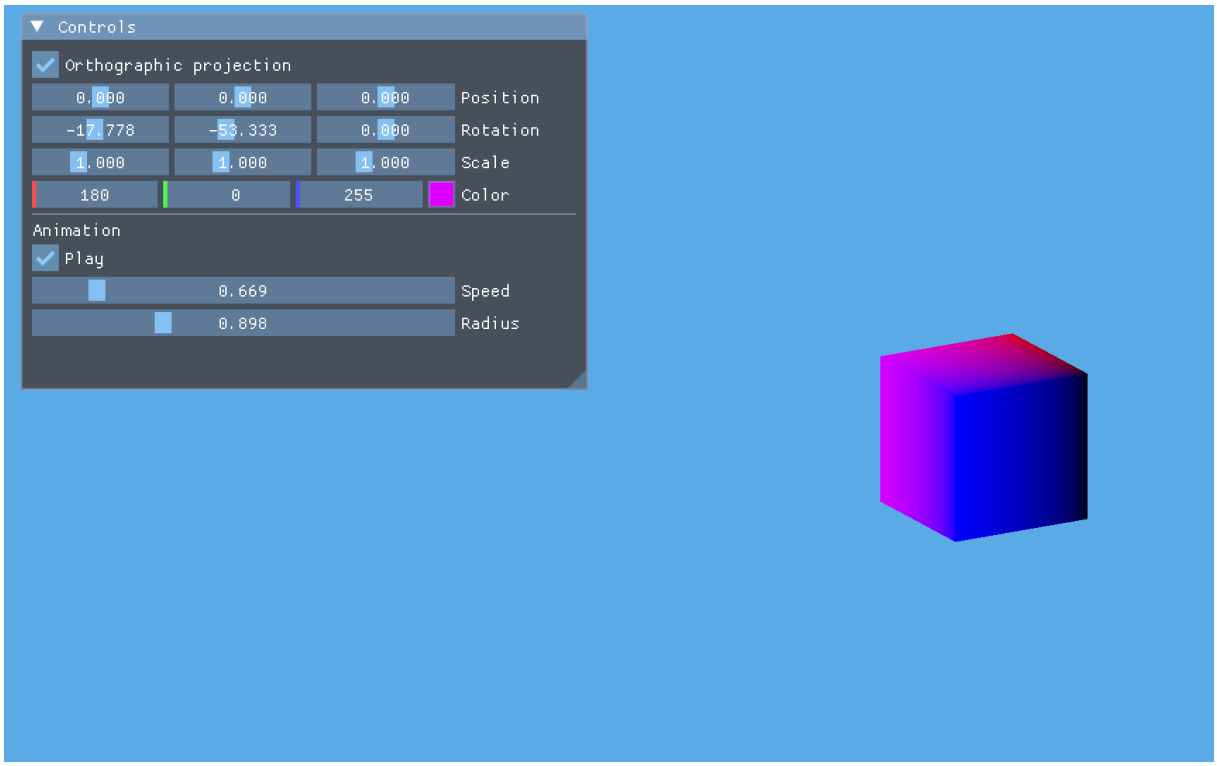
На рисунке показан результат работы программы. Слева расположено окно управления ImGui, содержащее чекбокс для переключения проекции, слайдеры трансформации (Position, Rotation, Scale) и блок анимации. Справа отображается вращающийся 3D-куб с процедурными цветами граней.

Скриншот 2. Ортографическая проекция и ColorEdit



При включении ортографической проекции (галочка установлена) перспективные искажения исчезают. С помощью палитры ColorEdit был изменен базовый цвет фигуры, который умножился на процедурные цвета вершин.

Скриншот 3. Анимация по траектории



Демонстрация работы анимации (включен чекбокс Play). Куб движется по круговой траектории с заданным радиусом и вращается вокруг своей оси. Скорость и радиус регулируются ползунками в интерфейсе.

Выводы

В ходе выполнения лабораторной работы я научилась настраивать графический конвейер (Graphics Pipeline) в API Vulkan. Были изучены способы передачи данных в видеопамять (вершинные и индексные буферы), а также методы передачи матриц трансформации через Uniform-буферы и наборы дескрипторов (Descriptor Sets).

Кроме того, я освоила математику 3D-графики: применение аффинных преобразований (сдвиг, поворот, масштабирование), разницу между перспективной и ортографической проекцией. Также был получен практический навык интеграции библиотеки пользовательского интерфейса ImGui для управления параметрами 3D-сцены в реальном времени (слайдеры, палитра цветов, чекбоксы) и создания процедурных цветовых эффектов в вершинных шейдерах.